



KubeCon



CloudNativeCon



OpenInfra
SUMMIT > ASIA



PyTorch
CONFERENCE

China 2026

#KubeCon #CloudNativeCon
#OpenInfraSummit #PyTorchCon

Declarative Underlays — Scaling Purpose-Built Infrastructure (Clusters) for OpenStack with Cluster API

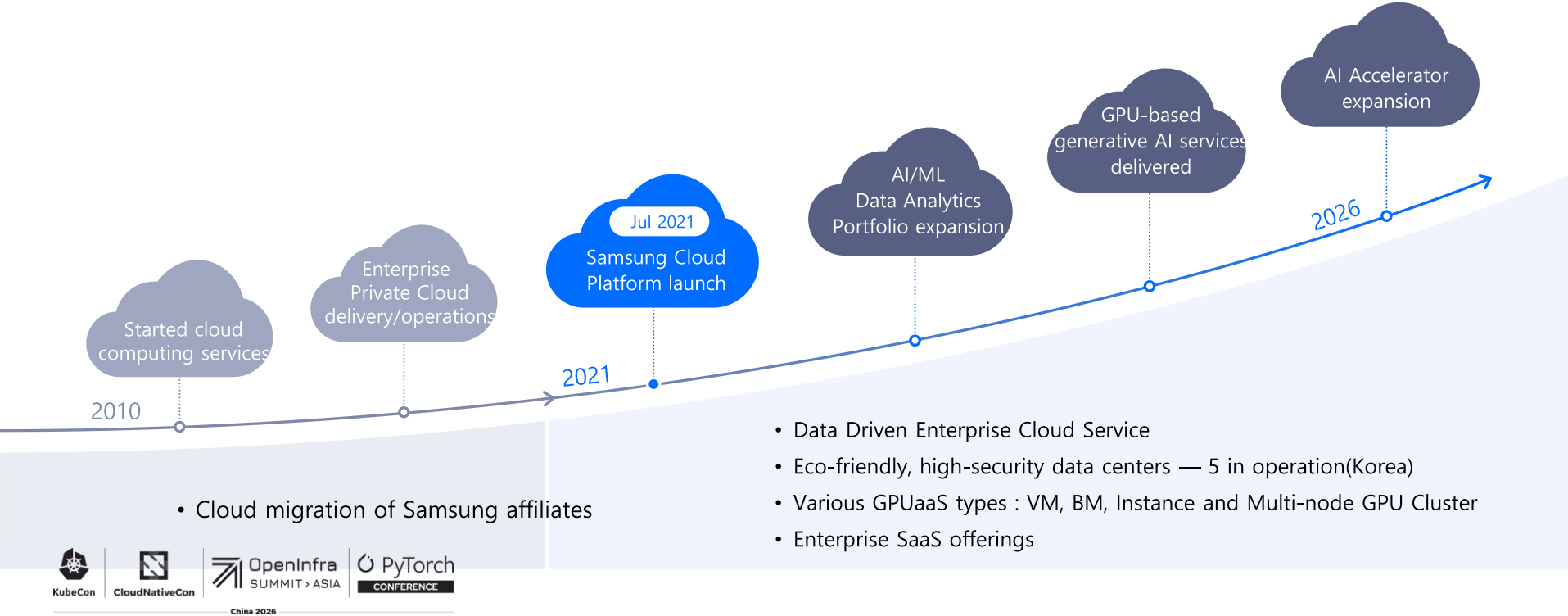
KubeCon + CloudNativeCon + OpenInfra Summit China 2026 | Shanghai, Sep 7-9

Hansol Park · Kangsub Song | Samsung SDS



Convenient and Reliable Cloud Service

10+ years of enterprise cloud service experience underpin the convenient, reliable Samsung Cloud Platform, accelerating our customers' business innovation.



Global Network

40 | Countries

55 | Subsidiaries/Branches(37/18)

17 | Cloud Data Centers

As of Q1 2026



Overseas: 10,770 (41.2%)

(incl. 3,700 global dev engineers)



KubeCon



CloudNativeCon

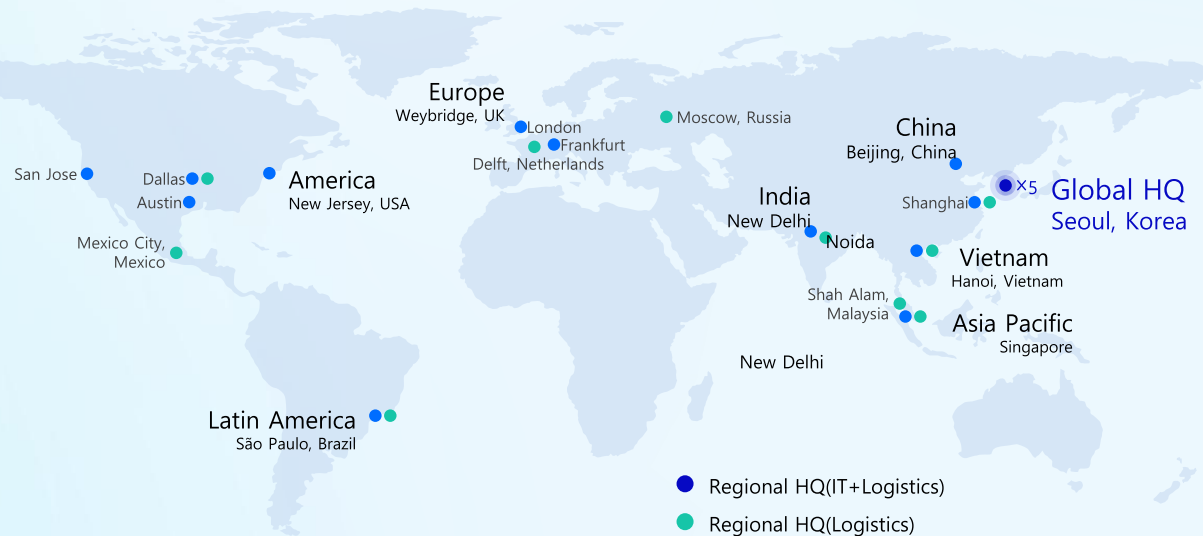


OpenInfra
SUMMIT > ASIA



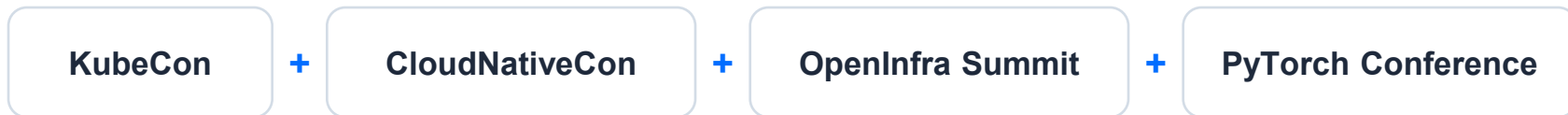
PyTorch
CONFERENCE

China 2026

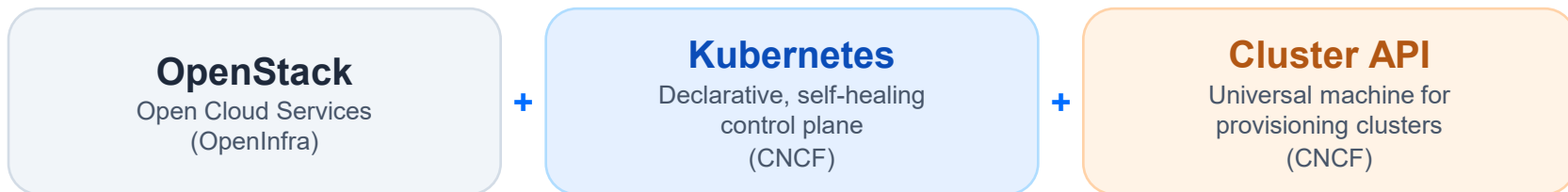


- Regional HQ(IT+Logistics)
- Regional HQ(Logistics)
- Cloud Data Center(Korea 5, overseas 12)

In the Spirit of the Conference



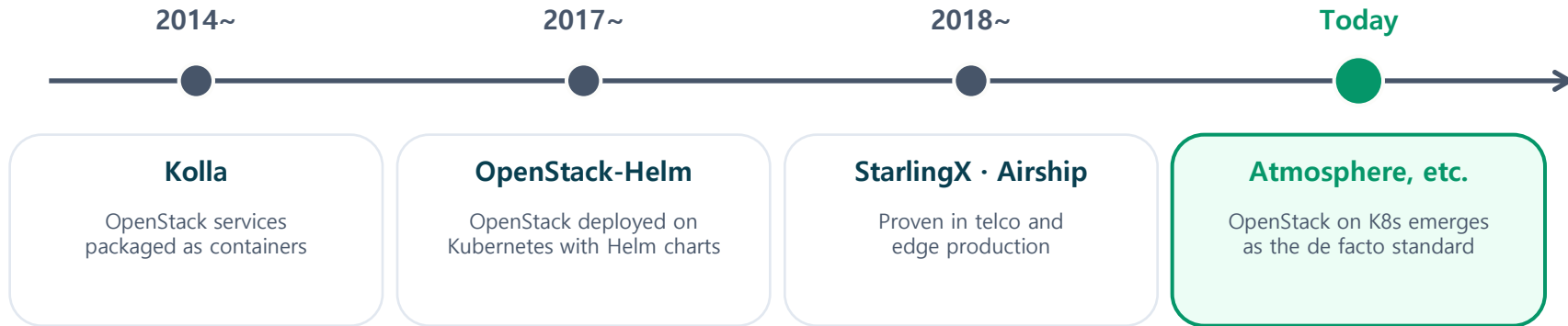
Four communities on one stage — open source thrives on convergence



Our talk follows the same spirit
OpenStack (OpenInfra) on Kubernetes (CNCF) /w Cluster API

OpenStack on Kubernetes: Not a New Topic

The community has been moving in this direction for nearly a decade



Problem: the Cluster Grows with Every New Business

GPU products

Dedicated nodes · drivers · latest K8s

High-performance networking

SR-IOV · dedicated NICs · packet perf

Storage products

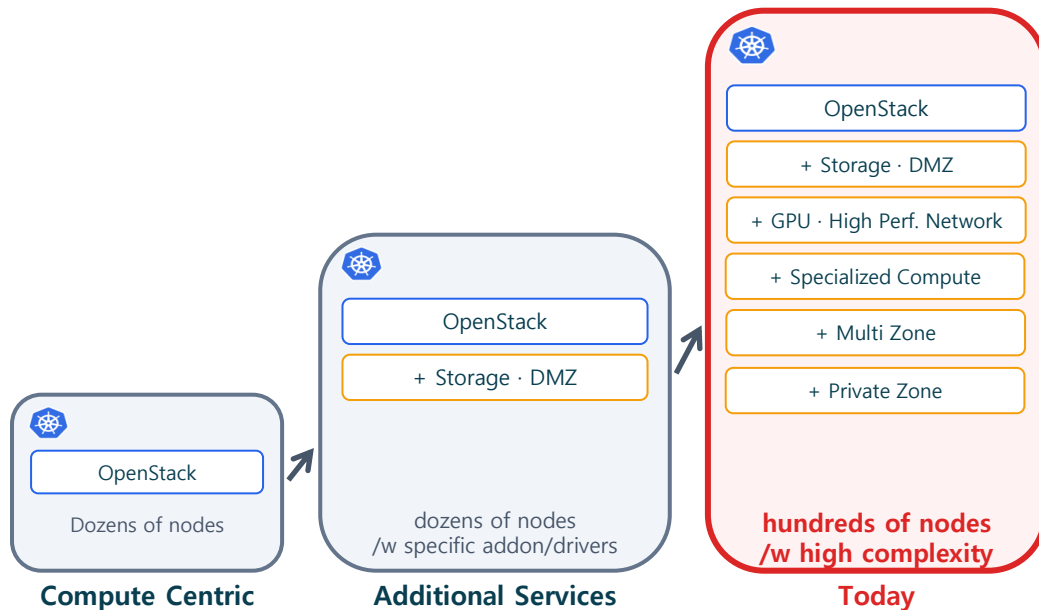
Dedicated CSI · high-volume I/O

DMZ · security products

Isolated auth & authorization

Users · Zone expansion

Ever-growing nodes & workloads



Cluster Scaling becomes a problem

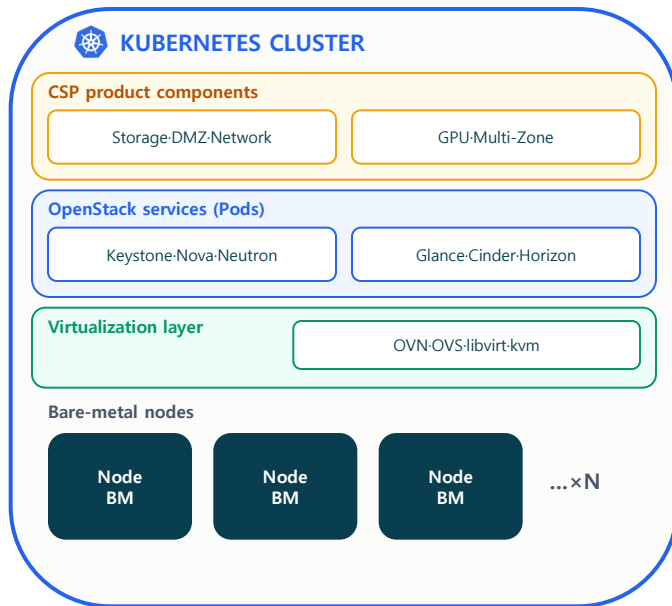
Where a Single Cluster Starts to Hurt

1 Components pile up

- Overlapping CNI · CRI · CSI
- Hundreds of CRDs
- Ever more host drivers
- Growing cross-product dependencies

3 Version management burden

- K8s & driver versions coupled
- One version bump = full revalidation
- Upgrades keep slipping



2 AuthZ boundary

- Hard to split Control-Compute-Service
- Isolation stops at namespaces
- Node level isolation

4 Failure boundary

- One workload's (data-plane) issue spreads
- If one goes down, all go down

Kubernetes is built for scale but...
real cost is the complexity operators have to absorb

The View from kubectl

```
$ kubectl get crds | wc -l
347

$ kubectl get crds | grep cluster
clusterclasses.cluster.x-k8s.io
clusterresourcesets.addons.cluster...
clusters.cluster.x-k8s.io          ← overlap
clusters.postgresql.cnpg.io        ← overlap

$ kubectl get cluster
# which one? CAPI vs CNPG
```

Hundreds of CRDs — two clashing 'cluster' resources

```
$ kubectl get pods -A -owide | grep node-01
multus-58x2k           Running   ← CNI
calico-node-x7k2       Running   ← CNI
ovn-controller-p9q1    Running   ← CNI
csi-cinder-node-4h8s   Running   ← CSI
csi-rbd-node-w2m5      Running   ← CSI

$ ls /opt/cni/bin
bridge calico multus ovn-k8s-cni...

$ crictl info | grep -i runtime
containerd / kata-containers ← CRI
```

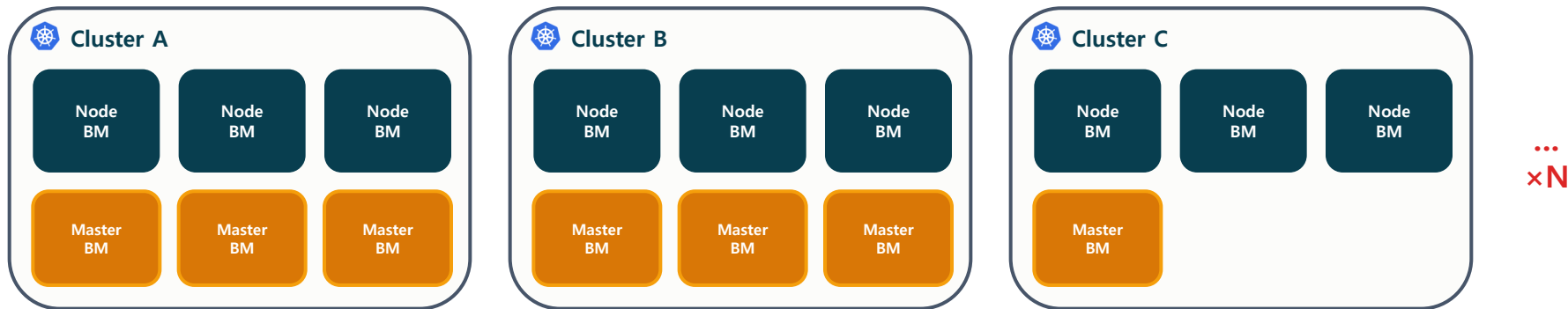
CNIs · CSIs · CRIs stacked on a single node

Our Approach

Split clusters by purpose

Let's keep clusters simple

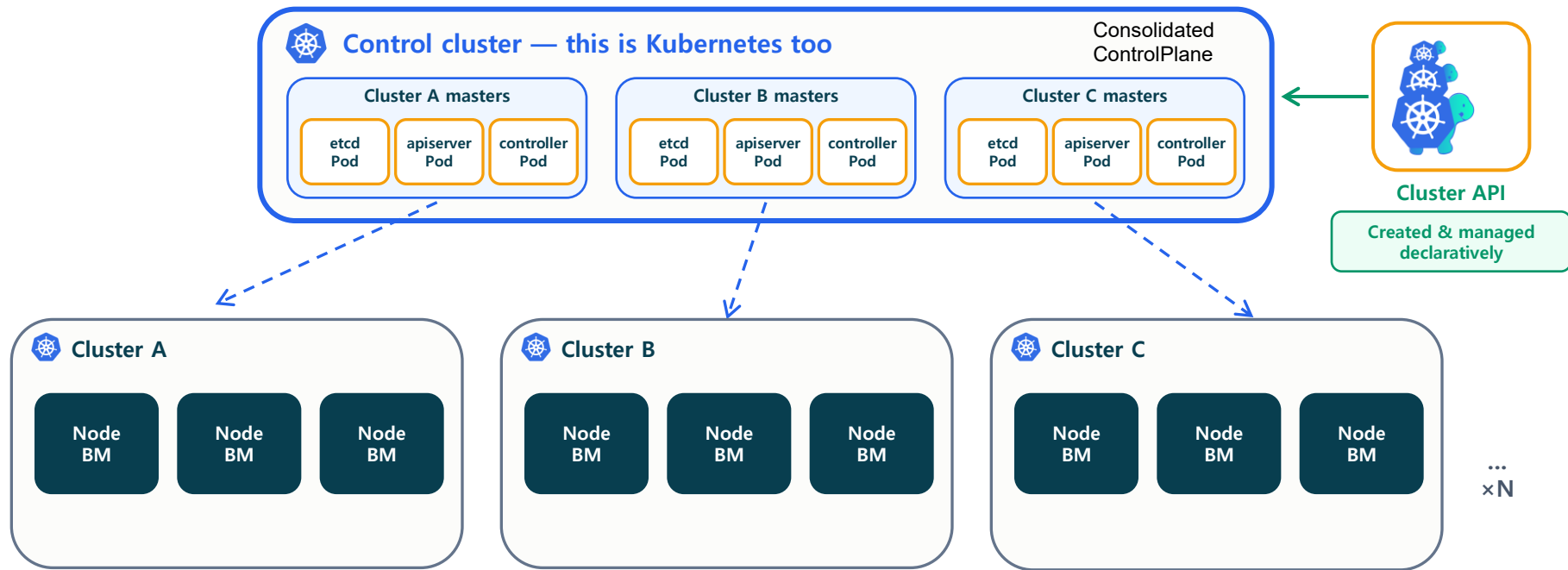
Cost of Splitting into Multiple Clusters



- × N **Control-plane lifecycle** — install, upgrade, patch every cluster's masters
- × N **Control-plane Resources** — Master Node per Cluster
- × N **Fleet management** — state, versions, health tracked per cluster
- × N **Copies of everything** — configuration, certificates, RBAC, monitoring

Splitting is the simple answer — but the price is everything × N

ClusterAPI to the Rescue



Redesign Principles

1. Node-level requirements stay isolated

Install drivers, runtimes, and special kernels only on the nodes that need them

2. Failures stop at the cluster boundary

Keep data-plane failures away from the management plane

3. Each service owns its technology stack

Choose Kubernetes, CNI, and CSI versions independently

4. Upgrades stay small and repeatable

Replace one cluster or node pool at a time



KubeCon



CloudNativeCon

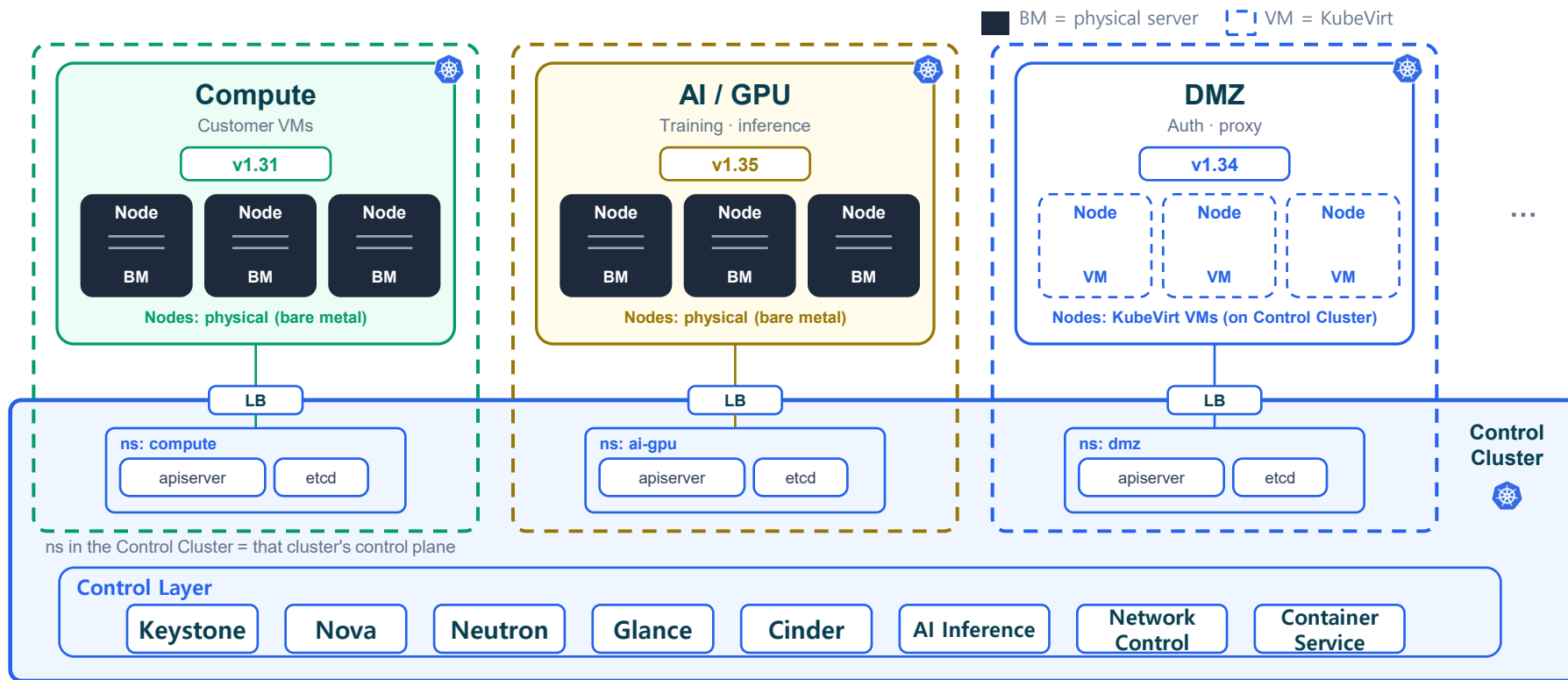


OpenInfra
SUMMIT > ASIA



PyTorch
CONFERENCE

Declarative Kubernetes — Fully Independent Structures



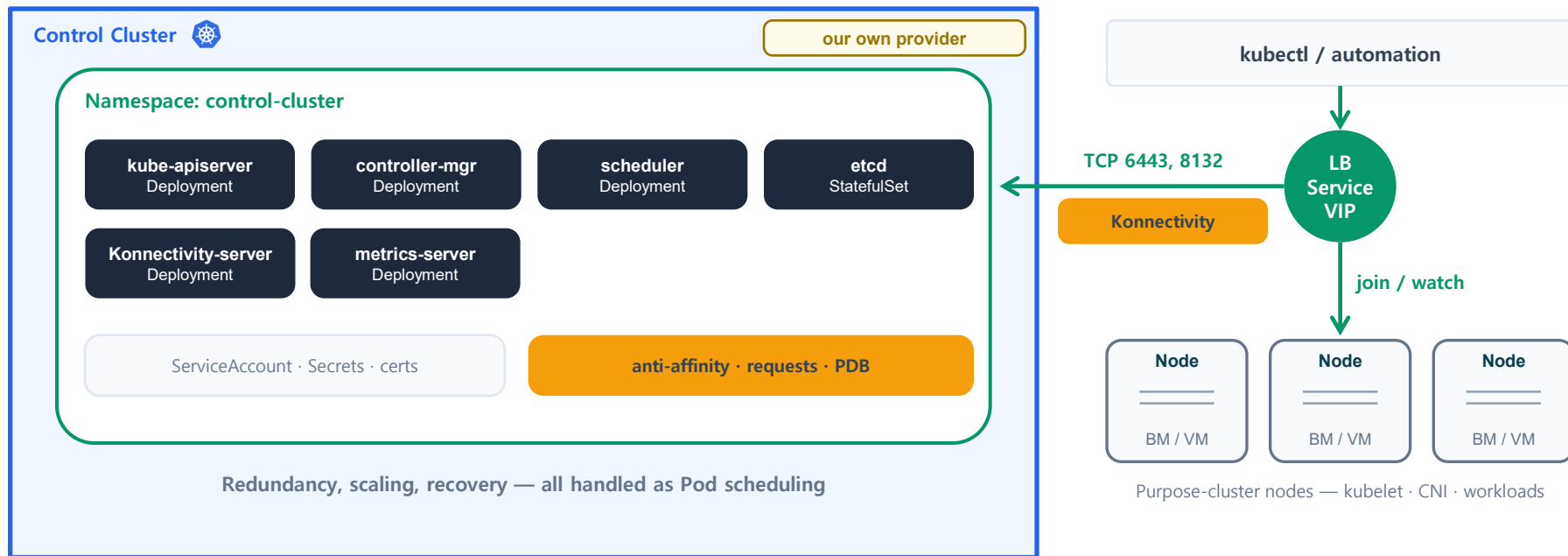


China 2026

Go deeper Technical Details



Inside the Control Cluster



Hosted control planes use control-cluster resources — worker capacity stays reserved for workloads



KubeCon



CloudNativeCon



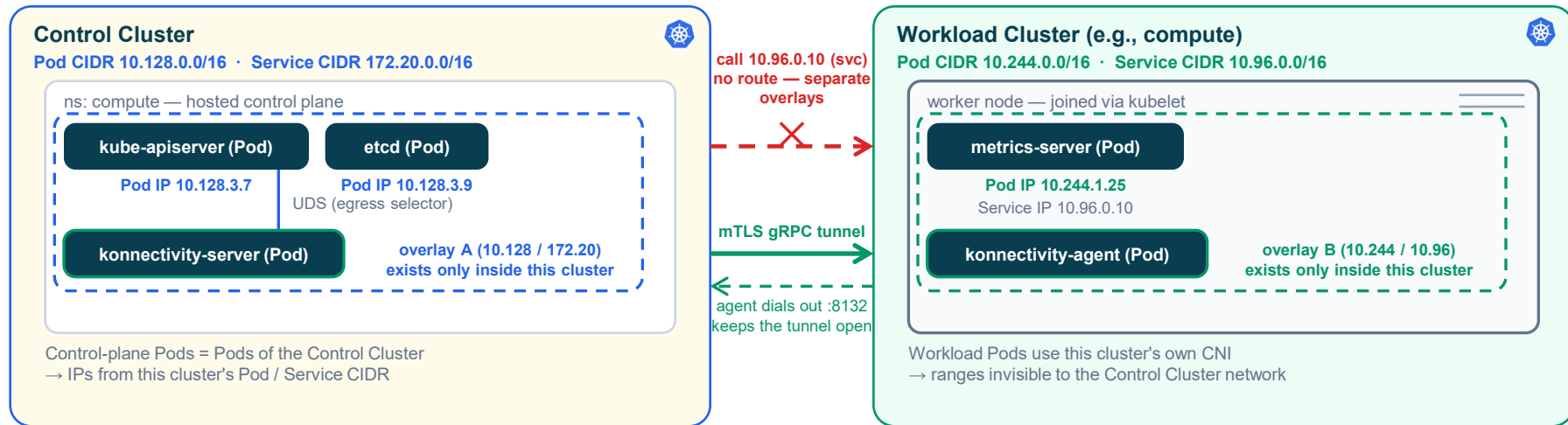
OpenInfra
SUMMIT > ASIA



PyTorch
CONFERENCE

Two Networks, One API Server — Konnectivity

kube-apiserver and metrics-server are Pods of two different clusters — each on its own independent overlay network



Metrics API: apiserver 10.128.3.7 (overlay A) → metrics-server 10.96.0.20 (overlay B) — each overlay exists only inside its own cluster, so the packet has nowhere to route

All API server → cluster traffic (aggregated APIs · webhooks · exec/logs) rides the Konnectivity tunnel



KubeCon



CloudNativeCon



OpenInfra
SUMMIT > ASIA

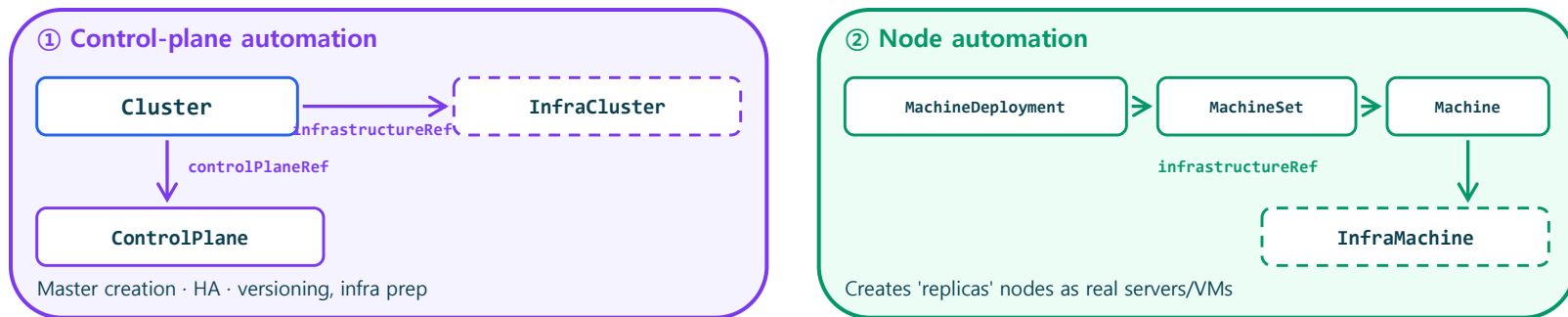


PyTorch
CONFERENCE

What Is Cluster API?

A CNCF project that declares the cluster itself as a Kubernetes resource

Declare a Pod, get a container — declare a Cluster, get a cluster



How it works: controllers keep comparing declared vs. actual state and converge them (reconcile)
Providers absorb infra differences (AWS, OpenStack, vSphere...) — an open contract: implement it

We built our own bare-metal underlay provider and control-plane provider



KubeCon



CloudNativeCon



CONFERENCE

Cluster Declaration in YAML

```
kind: Cluster
metadata:
  name: sample
spec:
  controlPlaneRef:
    kind: SCPControlPlane
  infrastructureRef:
    kind: SCPCluster
```

```
---
kind: SCPControlPlane
spec:
  version: v1.35.0
  podCidr: 10.244.0.0/16
  replicas: 3
```

```
---
kind: SCPCluster
spec:
  controlPlaneEndpoint:
    host: 192.0.2.10
```

```
kind: UnderlayMachineTemplate
metadata:
  name: sample-worker
spec:
  template:
    spec:
      underlayMachine:
        ...
---
kind: MachineDeployment
spec:
  clusterName: sample
  replicas: 3
  template:
    spec:
      infrastructureRef:
        kind: UnderlayMachineTemplate
        name: sample-worker
```

The values you actually touch

controlPlaneRef / infrastructureRef

- 1) Control Plane Provisioning
- 2) Infrastructure (Machine Node) Provisioning

version: v1.35.0

Change this value, and that's an upgrade

replicas: 3

- Control Plane – Number of Control planes (apiserver)
- Machine Deployment – Number of Nodes

underlayMachine:

Node form (bare metal / KubeVirt VM)

Reuse the template; change name, endpoint, and replicas

Every SCP* / Underlay* kind = our own CRD + controller — control plane as Deployment, etcd as StatefulSet, nodes via MachineDeployment



KubeCon



CloudNativeCon



OpenInfra
SUMMIT > ASIA



PyTorch
CONFERENCE

Four Things This Strategy Gave Us

Declarative clusters on Cluster API

①

Scale / Upgrade with Ease

- Scale = change replicas
- Create = one declaration
- Upgrade = image swap

②

Independent clusters and stacks

- Own version · CNI · CSI
- Separate security boundaries
- Failures stay in one cluster

③

Improve Resource Utilization

- Rebalance across clusters
- Absorb peaks without expansion
- Minimal idle time

④

Traceable, reproducible operations

- Method: IaC — shapes in Git
- History · review · rollback
- Humans & AI read and write

① Scale/Upgrade with Ease

Create, scale, and upgrade through one declarative workflow



KubeCon



CloudNativeCon

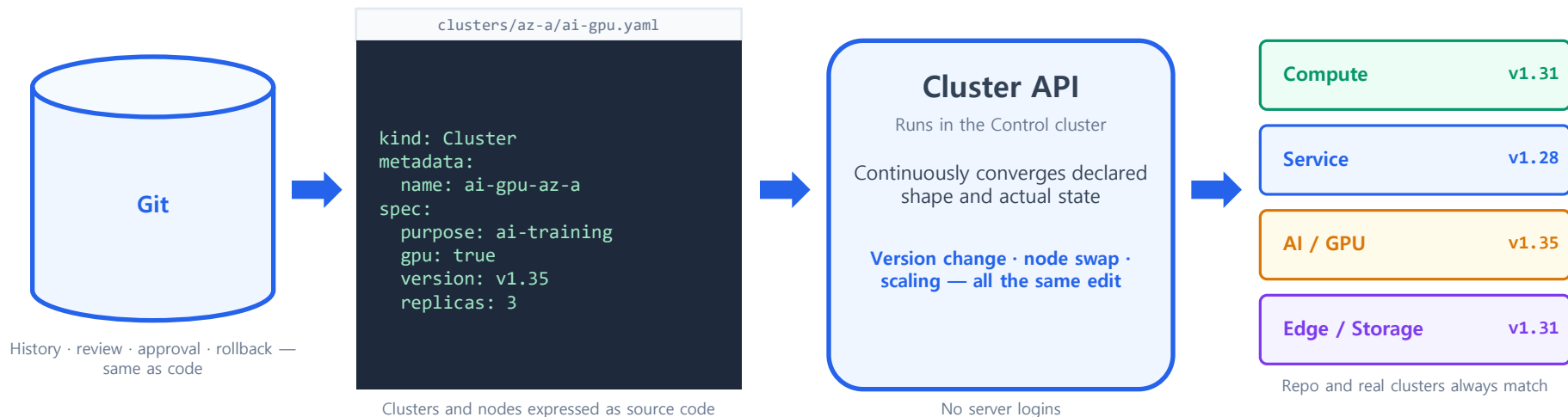


OpenInfra
SUMMIT > ASIA



PyTorch
CONFERENCE

Every Cluster Shape in One Git Repo



One source of truth keeps every cluster change visible through Git history

Cluster Creation Flow



Progress remains visible in Kubernetes object status; provisioning proceeds automatically

Usecase: Multi-AZ — Replicate the Same Architecture per AZ

Cluster API + GitOps — every AZ uses the same declarative model



```
clusters/  
├ az-a/  
├ az-b/  
└ az-c/
```

Add an az-c/ declaration
—
reuse the same blueprint

Availability Zone A

Service

Compute

AI / GPU

Edge / Storage

Same 4 purpose clusters

Availability Zone B

Service

Compute

AI / GPU

Edge / Storage

Same declaration · only sizes differ

Availability Zone C (new)

Service

Compute

AI / GPU

Edge / Storage

Add declaration to Git → auto-built

With capacity and networking ready, a new AZ reuses the same blueprint through Git

② Independent Clusters and Technology Stacks

Versions, technology stacks, and security boundaries separated per cluster



Usecase: Tech Stacks per Cluster

Cluster	Version	CNI	Scope	Additional Info.
Compute	v1.31	Cilium	Zonal	Prioritizes stability over latest features Customer VM
Hyper Network Service	v1.35	multus + ovs + SRIOV	Regional	Cluster for high-performance networking Network Drivers needed
AI / GPU	v1.35	multus + Cilium + SRIOV	Regional	Latest features · fast data access RDMA device plugin
Storage	v1.31	Calico	Zonal	Tuned for large sequential I/O
DMZ (Proxy)	v1.35	Cilium	Regional	Exposed zone — hardened network policy

③ Improve Resource Utilization

More work from the same fixed resource pool

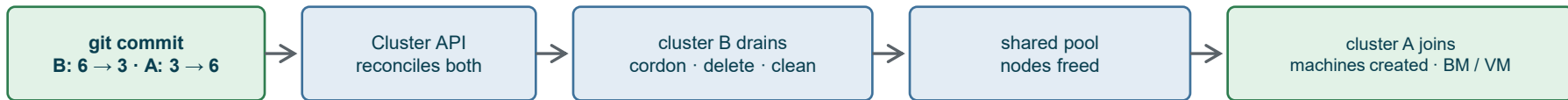
③ Improve Resource Utilization

Rebalance: Move Nodes from Idle to Busy

Why this matters for a CSP

Revenue comes from sold capacity — an idle node is paid-for hardware earning nothing. Utilization is the margin.

One commit rebalances — cluster B releases, cluster A claims



Release and claim are the same reconcile loop — minutes later, zero new hardware

Same fixed pool
zero new servers bought

Idle time ↓ = margin ↑
utilization is CSP profit

Both directions
one number · one commit

For a CSP, higher utilization turns existing hardware into available capacity



KubeCon



CloudNativeCon



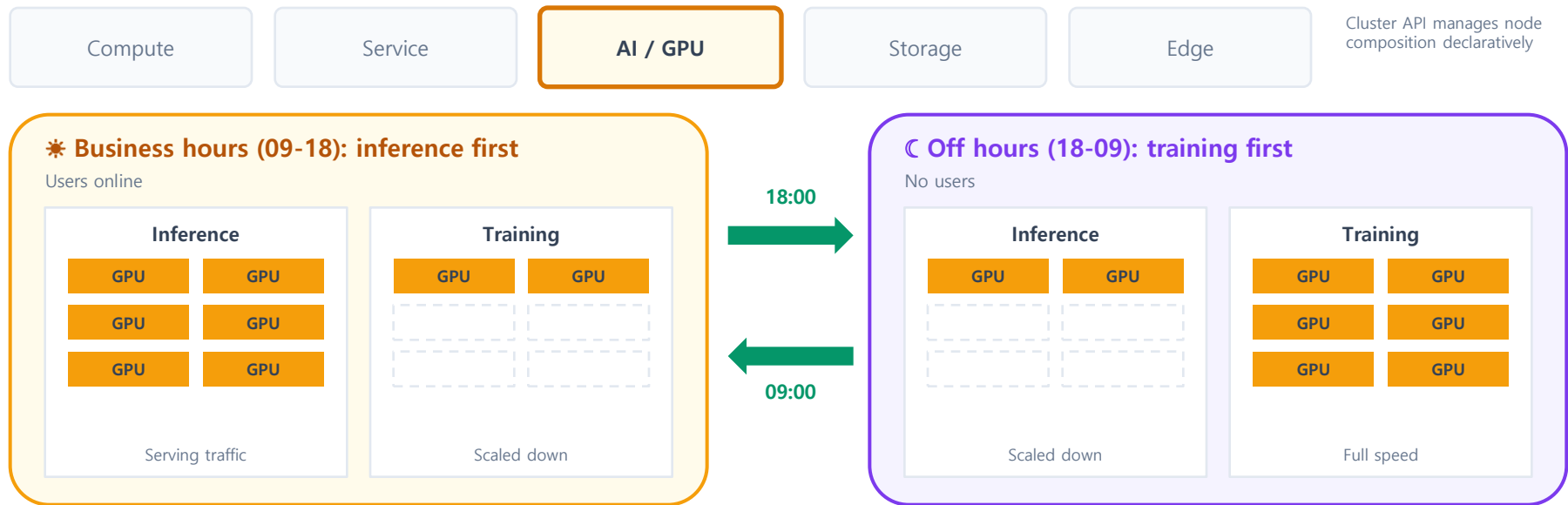
OpenInfra
SUMMIT > ASIA



PyTorch
CONFERENCE

③ Improve Resource Utilization

Usecase: Rebalancing GPUs by Time of Day



Each move is one declaration change — zero impact on other clusters

No expansion — just removing the idle hours of the same GPUs

④ IaC — Infrastructure as Code

Every infrastructure shape remains as code



KubeCon



CloudNativeCon



OpenInfra
SUMMIT > ASIA



PyTorch
CONFERENCE

Inside the Repo

The screenshot shows the GitHub repository for `samsung-cloud / cluster-infra`. The left sidebar displays the file structure, with the `az-a/` directory selected. The main content area shows the `compute` directory, which contains several YAML files. The `compute/md-standard.yaml` file is open, showing a `MachineDeployment` configuration for `compute-az-a-standard` with 24 replicas. The configuration includes a `spec` section with `machineType: std-v3`, `version: v1.31`, and `infrastructureRef` pointing to a `BareMetalMachineTemplate` named `std-v3-template`. Comments at the bottom of the file describe sibling pools for `md-highmem` and `md-highcpu`.

```
1 kind: MachineDeployment
2 metadata:
3   name: compute-az-a-standard
4 spec:
5   clusterName: compute-az-a
6   replicas: 24
7   template:
8     spec:
9       machineType: std-v3      # 16 vCPU · 64 GiB (highmem: mem-v3 · highcpu: cpu-v3)
10      version: v1.31          # Compute cluster · Cilium · zonal
11      infrastructureRef:
12        kind: BareMetalMachineTemplate
13        name: std-v3-template
14
15 # sibling pools in this cluster:
16 # md-highmem.yaml → mem-v3 (8 vCPU · 128 GiB) VM hosts, memory-bound
17 # md-highcpu.yaml → cpu-v3 (32 vCPU · 64 GiB) VM hosts, cpu-bound
```



KubeCon



CloudNativeCon

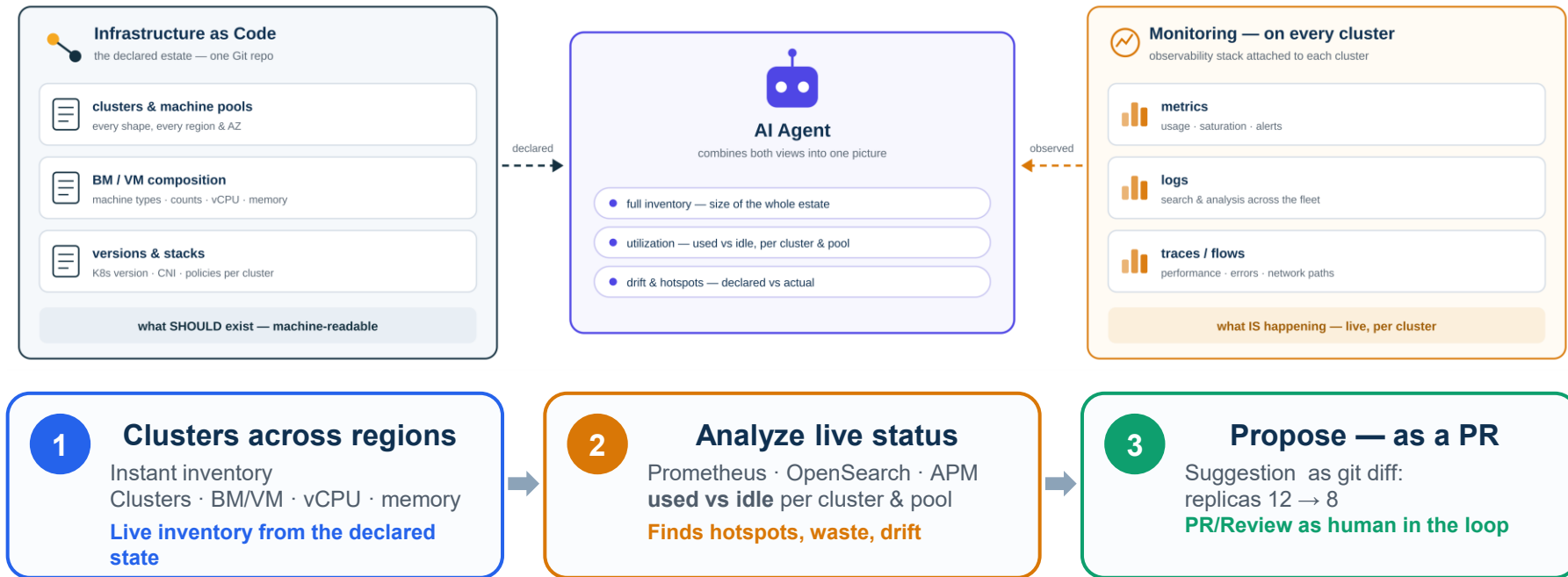


OpenInfra
SUMMIT > ASIA



PyTorch
CONFERENCE

Future Roadmap: AI Agents on Top of IaC



KubeCon



CloudNativeCon

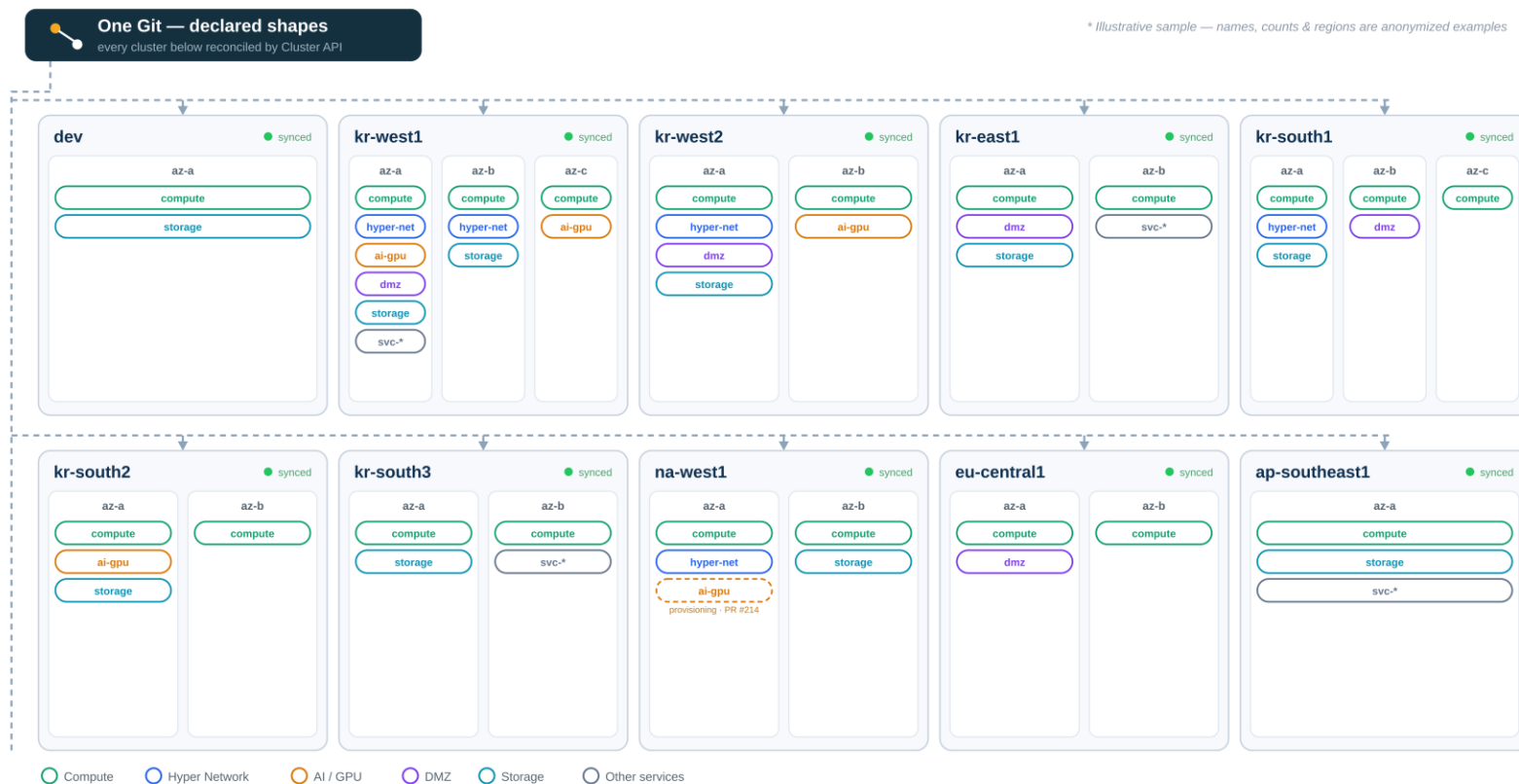


OpenInfra
SUMMIT > ASIA



PyTorch
CONFERENCE

Wrap-up — Conclusion



The Whole Infrastructure from One Terminal

```
mgmt-cluster · Cluster API · the whole fleet from one terminal

$ scpctl get clusters -A | head -14 # every region · zone · purpose, one command
NAMESPACE      NAME          CLUSTERCLASS  PHASE          VERSION  NODES  AGE
dev             compute-az-a  compute       Provisioned     v1.31    12     520d
kr-west1        compute-az-a  compute       Provisioned     v1.31    86     412d
kr-west1        compute-az-b  compute       Provisioned     v1.31    74     412d
kr-west1        hyper-net-az-a hyper-network Provisioned     v1.35    12     180d
kr-west1        ai-gpu-az-a  ai-gpu        Provisioned     v1.35    48     150d
kr-west1        dmz-az-a     dmz           Provisioned     v1.35    6      412d
kr-west2        compute-az-a  compute       Provisioned     v1.31    62     398d
kr-west2        ai-gpu-az-a  ai-gpu        Provisioned     v1.35    32     120d
kr-east1        compute-az-a  compute       Provisioned     v1.31    58     365d
kr-east1        dmz-az-a     dmz           Provisioned     v1.35    6      365d
na-west1        compute-az-a  compute       Provisioned     v1.31    44     201d
na-west1        hyper-net-az-a hyper-network Provisioned     v1.35    8      96d
na-west1        ai-gpu-az-a  ai-gpu        Provisioning    v1.35    -      4m    ◀ new cluster · PR #214
-               -            -             -              -        -      -      (+36 more)

49 clusters · 9 regions + dev · 20 AZs — one command, no SSH, no per-cluster tooling

$ git log --oneline -1 && git diff HEAD~1 # scaling a pool = a one-line commit
9f3c2e1 scale compute standard pool 18 ~ 24 (hansol.park, 2 minutes ago)
--- a/clusters/kr-west1/az-a/compute/md-standard.yaml
+++ b/clusters/kr-west1/az-a/compute/md-standard.yaml
- replicas: 18
+ replicas: 24

$ scpctl -n kr-west1 get machinedeployment compute-az-a-standard -w
NAME                CLUSTER    REPLICAS  READY  UPDATED  PHASE
compute-az-a-standard compute-az-a 24       21     24       ScalingUp
compute-az-a-standard compute-az-a 24       24     24       Running
# controllers converge declared ~ actual on their own — merge the commit, done ✓
```



Conclusion: Why This Route — and What It Gave Us

Why we took this route

Large scale

Heterogeneous bare-metal & VM infrastructure offered as service across products

Constant growth

New regions, AZs, and services
Spent weeks on manually installing underlay clusters

The old way broke

Hand-built clusters:
Slow scale-out / Risky upgrades /
Management Price

We needed an architecture designed for scale

What it gave us

① **Scale & upgrade with ease**

② **Independent clusters**

③ **Higher utilization**

④ **Everything as code**

Treat clusters as declarative, replaceable infrastructure — not handcrafted environments



China 2026

Thank You · 谢谢

Questions? | Kangsub Song · Hansol Park,
Samsung SDS